



3

STATIC AND DYNAMIC CODE ANALYSIS



While initial triage, automated sandboxing, and behavioral analysis are essential steps in understanding an unknown malware sample, sometimes you need to dig deeper, down to the code level. Maybe you're experiencing problems getting the sample to run in a sandbox or VM environment, or perhaps you're trying to identify any hidden capabilities. Whatever your reasons, this chapter will walk you through static and dynamic code analysis techniques you can use to reverse engineer a malware sample and discover its true intentions.

We'll start with a brief introduction to assembly code, a fundamental concept for reverse engineering PE files. Then we'll dig into static code analysis and disassemblers like IDA. Finally, we'll explore the details of dynamic code analysis and debugging with x64dbg.

NOTE

As with **Chapter 2**, the goal of this chapter is to introduce key concepts that will be referenced later in this book. It's not meant to be a comprehensive guide to these techniques, but you can find some great beginner's resources listed in **Appendix C**.

Introduction to Assembly Code

Assembly is a low-level programming language that provides a human-readable representation of machine code instructions. When reverse engineering malware, malicious programs can be converted from binary machine code to assembly code; a process referred to as *disassembly*.

This section introduces x86 (32-bit) and x86_64 (64-bit, referenced as x64 from here on) assembly code and some CPU concepts that you'll apply throughout the rest of the book. We'll start with CPU architecture basics and then move on to assembly instructions.

CPU Registers

As a program is running, CPUs use *registers*, which are memory locations on the physical processor chip, to store data and keep track of the processing state. Because memory storage is much slower, the CPU takes advantage of registers as much as possible for data storage and manipulation. Depending on processor architecture, each register can store a certain amount of data. A *word* is equal to 16 bits of data. An x86 processor register usually can store one *dword* (32 bits) of data, while an x64 processor register can store one *qword* (64 bits) of data.

There are five primary types of CPU registers: (1) general registers, (2) index and pointer registers, (3) flag registers, (4) segment registers, and (5) indicator registers. The first three are the most important for our purposes here, but I'll touch on the other two later in the book.

General Registers

General registers are used to store and process data for general purposes such as arithmetic operations and function arguments. Each general register can be split into smaller segments containing 16 or 8 bits of data. For example, the x64 RAX register, which can store 64 bits of data, “contains” four additional smaller general registers: EAX (the last 32 bits of data in RAX), AX (the upper 16 bits of EAX), AH (the upper 8 bits of EAX), and AL (the lower 8 bits of EAX). **Figure 3-1** shows the RAX register and its smaller segments with their respective storage size limits in bits.

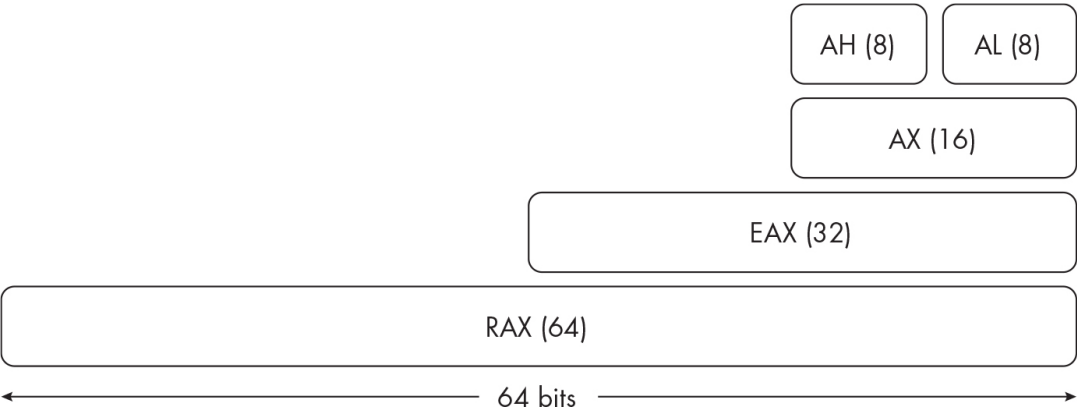


Figure 3-1: General-purpose register layout

Table 3-1 describes each general register for x86 and x64 processors. Note that these descriptions reflect how each register has been used historically; this doesn’t mean that the register *must* be used in this way.

Table 3-1: x86 and x64 General Registers

x86 regis-ter	x64 register	Description
EAX	RAX	The accumulator register, used for tasks such as arithmetic, interrupts, and storing return

x86 regis- ter	x64 register	Description
values		
AX, AH, AL	Same as x86	Upper 16 bits of EAX, upper 8 bits of EAX, and lower 8 bits of EAX, respectively
EBX	RBX	Used for referencing variables and arguments
BX, BH, BL	Same as x86	Upper 16 bits of EBX, upper 8 bits of EBX, and lower 8 bits of EBX, respectively
ECX	RCX	The counter register, used for counting and loop control
CX, CH, CL	Same as x86	Upper 16 bits of ECX, upper 8 bits of ECX, and lower 8 bits of ECX, respectively
EDX	RDX	The data register, used primarily for arithmetic operations and sometimes as a backup for EAX
DX, DH, DL	Same as x86	Upper 16 bits of EDX, upper 8 bits of EDX, and lower 8 bits of EDX, respectively

Index and Pointer Registers

Index registers and *pointer registers* can store both pointers and addresses. They can be used for tasks such as transferring memory data, maintaining control flow, and keeping track of the stack. **Table 3-2** provides an overview of these registers.

Table 3-2: x86 and x64 Index and Pointer Registers

x86 regis- ter	x64 regis- ter	Description
ESI	RSI	The source index; typically serves as the source address in memory operations
EDI	RDI	The destination index; typically serves as the destination address in memory operations
EBP	RBP	The base pointer; points to the base of the stack
ESP	RSP	The stack pointer; points to the last item pushed to the stack
EIP	RIP	The extended instruction pointer; points to the address of the code that will be executed next

The Flags Register

The *flags register* keeps track of the current state of the processor. Generally, it's used for storing the results of computations and controlling the processor's operation. *Flags* is a general term for the EFLAGS register, which is used in 32-bit architectures and shown in [Figure 3-2](#), and the RFLAGS register, which is used in 64-bit architectures. These two registers function similarly to each other.

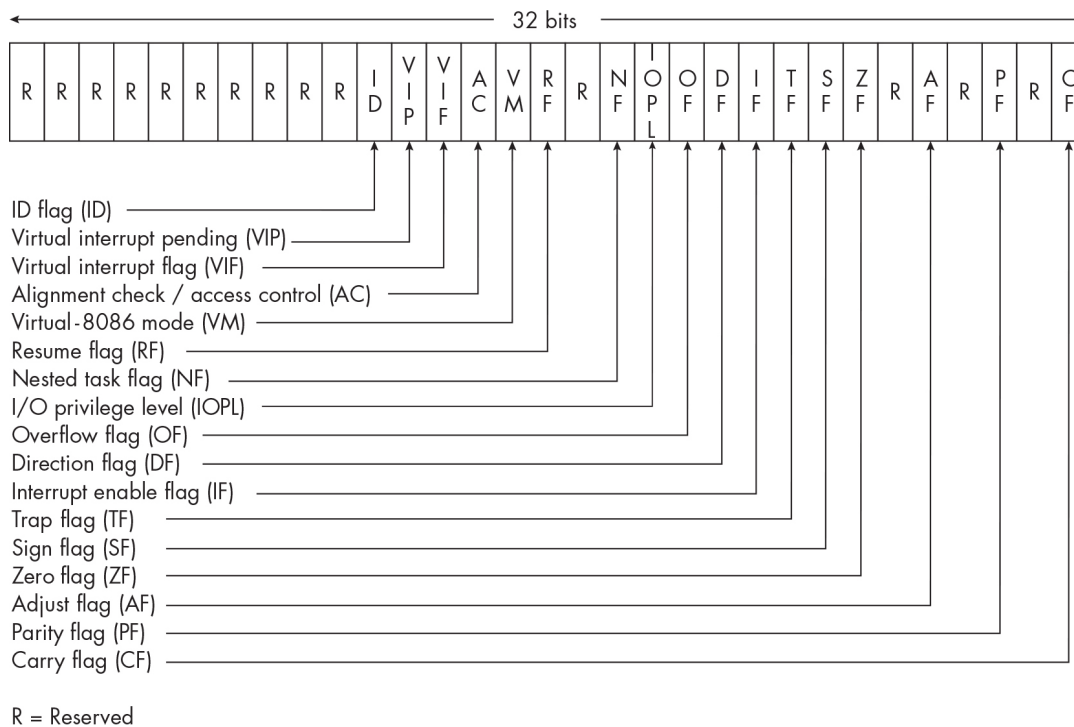


Figure 3-2: The EFLAGS register

The two most important flag values for our purposes are the *zero flag* (ZF) and the *trap flag* (TF). The ZF is a single bit in length and is set with a conditional instruction. For example, a conditional instruction may compare two values; if the values are the same, the ZF will be set to 1. The TF is used for debugging purposes and allows the debugger to single-step through instructions.

X86 VS. X64 CPU ARCHITECTURES

You may have already noticed slight differences between the x86 and x64 processor architectures. The CPU registers use different naming conventions (RAX versus EAX, for example), and x64 registers can store 32 bits more data than their x86 counterparts. One other difference is that the x64 instruction set can use the additional registers R8 through R15, which act as general-purpose registers. Because using processor registers for data storage and manipulation is much faster than accessing memory, 64-bit processors have a clear speed advantage. In assembly code, you'll likely see x64 programs using CPU registers for data storage and manipulation directly, rather than using both memory and CPU registers in concert.

Because of these extra registers, it is common for 64-bit programs (including malware) to pass parameters to functions via CPU registers (as opposed to pushing parameters to the stack in 32-bit programs). We'll touch on this again a bit later in this chapter.

x64 and x86 Instructions

Now that we've covered the basics of CPU registers and the stack, let's start digging into the various assembly instructions available for a program to use.

Stack Operations

Chapter 1 briefly mentioned the stack, a region of memory assigned to a thread and used to store temporary data such as variables, pointers, and other objects that will no longer be needed after the thread completes execution and is terminated. The stack operates in a *last-in, first-out (LIFO)* way. This means that when a program stores data, let's say a variable, on the stack, that variable goes to the top of the stack. To retrieve the variable, the program must first retrieve all the other data above it.

To place data on the stack, the program executes a `push` instruction, which pushes the data to the top of the stack (see **Figure 3-3**). To retrieve that data, the application executes a `pop` instruction, which pops the data off the top of the stack.

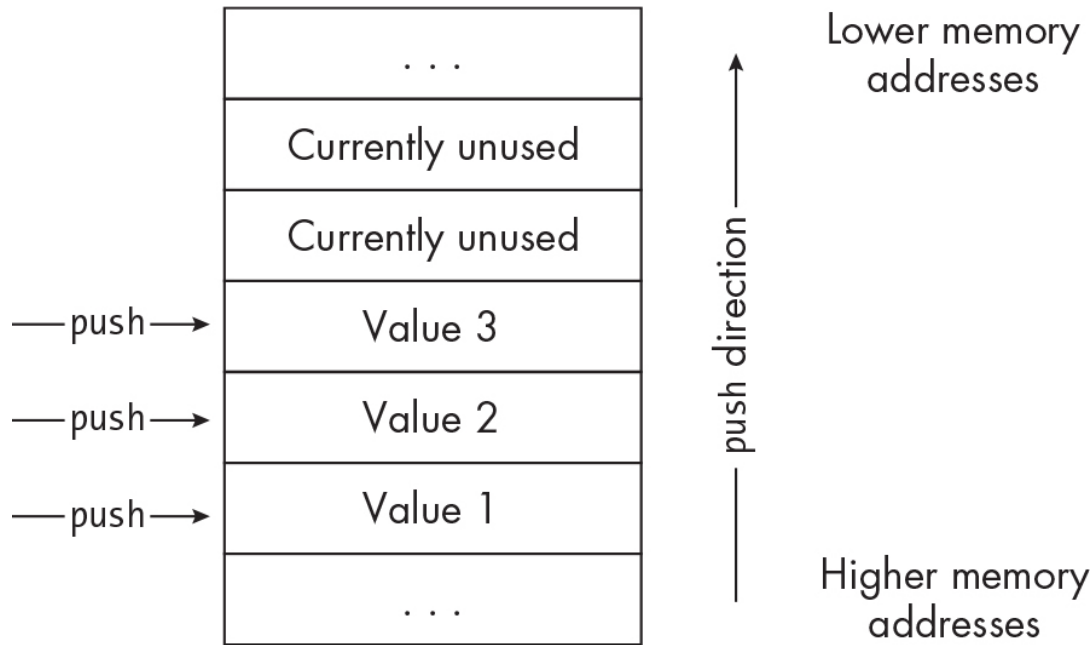


Figure 3-3: A program pushing data to the top of the stack

Here the program is pushing Value 1 onto the stack first, followed by Value 2 and then Value 3. Value 3 is now at the top of the stack. To retrieve Value 1, the program will have to pop Values 3 and 2 off the stack first, in that order.

Table 3-3 provides an overview of these instructions.

Table 3-3: Stack Operation Instructions

Instruction	Example	Description
<code>push arg1</code>	<code>push ebx</code> <code>push [ebx]</code> <code>push "50"</code>	Push (store) the data in <i>arg1</i> (which can be a register, memory address, or constant) to the top of the stack.
<code>pop arg1</code>	<code>pop ebx</code> <code>pop [ebx]</code>	Pop (retrieve) data from the top of the stack and stores it in <i>arg1</i> , which can be a register or memory address.

Most instructions can operate on CPU registers directly as well as on memory addresses. For example, the `push ebx` instruction will push whatever data is currently stored directly in the EBX register onto the stack. Brackets around a register name, such as `push [ebx]`, indicate that the instruction is dereferencing a pointer to memory, so whatever is stored inside the memory address in EBX will be pushed to the stack. For example, if EBX currently contains the value `0x00406028` (a memory address), whatever is currently stored at that memory address will be pushed onto the stack. In a disassembler (discussed shortly), you'll usually see this instruction written as `push byte ptr [ebx]` or similar, giving you a clue that this is a pointer to a sequence of bytes in memory.

Arithmetic Operations

Data manipulation and arithmetic instructions are used for computations like sums and differences. Some arithmetic instructions, such as `add`, take two arguments: the first is a target operand and the second is the value to add to it. Others, like `dec`, which decrements a target operand, take only one argument. **Table 3-4** summarizes some common arithmetic operation instructions.

Table 3-4: Arithmetic Operation Instructions

Instruction	Example	Description
<code>add arg1, arg2</code>	<code>add ebx, 50</code>	Add the value of <i>arg2</i> (a register, memory address, or constant such as the value 50) to <i>arg1</i> (a register or memory address).
<code>sub arg1, arg2</code>	<code>sub ebx, 50</code>	Subtract the value of <i>arg2</i> (a register, memory address, or constant) from <i>arg1</i> (a register or memory address).

Instruction	Example	Description
<code>inc arg1</code>	<code>inc ecx</code>	Increment <i>arg1</i> (a register or memory address) by 1.
<code>dec arg1</code>	<code>dec ecx</code>	Decrement <i>arg1</i> (a register or memory address) by 1.

Data Movement

A program can move data to and from memory and registers with the `mov` instruction. The `mov` instruction takes two arguments, but only one can be a memory address. For example, in x86 and x64 assembly code, a program can't move data directly from a memory address to another memory address. You can see some common examples of these instructions in [Table 3-5](#).

Table 3-5: Examples of the `mov` Instruction

Instruction	Example	Description
<code>mov arg1, arg2</code>	<code>mov eax, ebx</code> <code>mov [ebx], 100</code>	Move the data in <i>arg2</i> (a register, memory address, or constant) to <i>arg1</i> (a register or memory address).

Value Comparisons

Quite often, a program will need to compare two values to direct control flow. Comparison instructions may be `if` statements, such as `if var == 2`, but the two primary comparison instructions are `cmp` and `test`. The result of either instruction is stored in the zero flag register, which will later be used to direct control flow. [Table 3-6](#) provides an overview of the `cmp` and `test` instructions.

Table 3-6: Comparison Instructions

Instruction	Example	Description
<code>cmp arg1, arg2</code>	<code>cmp eax, ebx</code> <code>cmp eax, 5</code>	Compare <i>arg1</i> (a register or memory address) with <i>arg2</i> (a register, memory address, or constant).
<code>test arg1, arg2</code>	<code>test eax, ebx</code> <code>test eax, 5</code>	Same as above.

You may see instructions such as `test eax, eax` that are comparing the value in EAX to itself. This is simply a way of checking to see whether the content of the register (EAX, in this case) is 0. When both arguments are the same in a `test` instruction, it's essentially comparing the argument to 0. If EAX is 0, the zero flag will be set.

Though `cmp` and `test` look incredibly similar, there are fundamental differences in how they work: `cmp` can be seen as a `sub` instruction, and `test` is similar to the `and` instruction. The full details are beyond the scope of this chapter.

Jump Instructions

A program can use various forms of jump instructions to skip to another area of code or to modify control flow based on the comparison instructions just described. There are three common types of jump statements worth noting for our purposes, summarized in [Table 3-7](#).

Table 3-7: Jump Instructions

Instruction	Example	Description
<code>jmp <i>arg1</i></code>	<pre> jmp func_00405207 jmp ebx </pre>	The “jump” instruction: jump to another address, function, or segment of code; <i>arg1</i> can be a register (containing a memory address), a pointer, or an address in code.
<code>jz <i>arg1</i></code>	<pre> jz func_00405207 jz ebx </pre>	The “jump-if-zero” instruction: jump to <i>arg1</i> if the last arithmetic operation resulted in 0.
<code>jnz <i>arg1</i></code>	<pre> jnz func_00405207 jnz ebx </pre>	The “jump-if-not-zero” instruction: jump to <i>arg1</i> if the last arithmetic operation did not result in 0.

In the case of the conditional jump statements (such as `jz` and `jmp`), these instructions inspect the zero flag register for input. Because `cmp` and `test` set these flags, they’re usually a precursor to conditional jumps.

Call and Return Instructions

Programs issue call instructions to invoke a Windows API function or to skip to a target function in the code. In the latter form, `call` works a lot like an unconditional jump instruction. Before jumping to a new address, `call` instructions push the current address (stored in EIP or RIP) to the stack. Later, the program can issue a return (`ret`) instruction to go back to the previous location in code. [Table 3-8](#) describes these instructions.

Table 3-8: Call and Return Instructions

Instruction	Example	Description
<code>call arg1</code>	<code>call ebx</code> <code>call</code> <code>WriteFile</code>	Call (or jump to) the address in <i>arg1</i> , which can be a register (containing a memory address), pointer, or function.
<code>ret</code>	<code>ret</code>	Return to the previous code location before the <code>call</code> instruction executed.

No-Operation Instructions

No-operation, or `nop`, instructions do exactly what you'd think: nothing. An address with a `nop` instruction is essentially skipped over by the processor. If you're wondering what their purpose is, `nop` instructions are used for various legitimate reasons, including alignment of code and memory, timing purposes (such as testing a program's execution speed), and placeholder code (in manual assembly programming, for example).

However, `nops` can also be used for more nefarious purposes, such as in shellcode (discussed in [Chapter 12](#)) and exploit code (such as buffer overflows). The presence of `nop` instructions in an assembly code segment can be a good signal to an analyst that there's something worth investigating further.

Now that we've looked at the basics of assembly code, let's turn our attention to investigating malicious code via static code analysis.

Static Code Analysis

Static code analysis is the technique of inspecting code in its static state (that is, not during execution) and is usually accomplished with a

tool called a *disassembler*. Disassemblers allow us to navigate through the malware's code, identify functions or code blocks of interest, and dive deeper into those areas. Knowing how to use disassemblers effectively is often what separates novice malware analysts from intermediate and advanced ones. Let's say you have an unknown executable file that exhibits only a few behaviors in your automated sandboxes and VMs, or perhaps it doesn't even run properly. Maybe it's using some VM detection and sandbox evasion techniques. Novice malware analysts might throw in the towel at this stage. Experienced analysts, however, would likely load the sample into a disassembler to identify where to focus their investigation efforts next.

DISASSEMBLY

HOW IT WORKS

Disassembly is the first step in malware code analysis. When a binary file is run through a disassembler, the file is essentially broken down into an assembly code representation of the machine code. Different disassemblers break the code down according to their own algorithms, in one of two ways:

Linear Disassembly (aka Linear Sweeping)

Linear disassemblers begin with the first bytes of the file, disassemble those, and then sweep through the rest of the file, disassembling the code along the way. This method, often used by debuggers, is quick and efficient but more prone to errors than recursive disassembly.

Recursive Disassembly (aka Flow-Oriented Disassembly)

Like linear disassemblers, recursive disassemblers begin to disassemble the code byte by byte, but they pause when they encounter a flow control sequence, such as an `if... then` statement or a loop. The disassembler follows the logic of the code and can even simulate the execution of condi-

tional branches, similarly to how the program would execute if it were actually running. This method, used by disassemblers such as IDA and Ghidra, produces a better representation of disassembled code, but it is much slower than linear disassembly and can still be prone to certain errors.

THE CODE VS. DATA PROBLEM

When a disassembler disassembles a program, it's essentially making educated guesses about how to deconstruct the code. Disassemblers can interpret the raw bytes of an assembled program in multiple ways: as assembly instructions, hexadecimal data, strings, and so on. This presents a problem for disassemblers and reversers, as sometimes it's not clear how that raw data was meant to be interpreted and executed by the CPU at runtime. Modern disassemblers do a competent job at inferring this information, but they can sometimes fall short, as you'll see in [Chapter 9](#).

The takeaway here is that when you notice anomalies or unintelligible instructions in disassembled code, what you're looking at may not be code at all; the disassembler may have incorrectly disassembled the data. The opposite could be true as well: the disassembler may misidentify data that *is* supposed to be code. We'll revisit this topic in later chapters.

Choosing a Disassembler

Two of the best-known disassemblers in use today are IDA and Ghidra. Both are interactive disassemblers, meaning you can interact with and manually manipulate the disassembled code. This allows you to modify code, add comments, rename functions, fix incorrectly disassembled code, and in general have greater control over the reverse engineering process.

IDA (<https://hex-rays.com>) has always been one of the most popular interactive disassemblers, for good reason. It can disassemble many

different file types, has support for many different processor architectures and operating systems, and features many plug-ins and scripts created by Hex Rays and the community. IDA comes in various flavors but offers a free version that is sufficient for many reverse engineering scenarios.

Ghidra, on the other hand, is a completely free and open source interactive disassembler that implements many of the features of IDA and some additional ones such as collaborative disassembly, allowing multiple people to work on a single file. Ghidra is newer and, at the time of this writing, doesn't have as many plug-ins or extensibility scripts available as IDA. These will come with time, however. You can find a curated list of helpful IDA and Ghidra plug-ins and scripts at <https://github.com/fr0gger/awesome-ida-x64-olly-plugin>.

There are many fans in both the IDA camp and the Ghidra camp, but ultimately which one you choose doesn't matter much. Once you grasp assembly concepts, either option will get the job done. For this chapter (and throughout most of this book), I'll be using IDA.

Analyzing with IDA

Let's walk through the basic process of static code analysis with IDA.

NOTE

In this section, we'll be investigating a malware file in IDA that you can download from VirusTotal or MalShare using the following file hash:

SHA256:

30c9a1460615254a4ef791ffeeba53f4a31361c2646b25d3909025178c5a3976

To open this suspicious file in IDA, navigate to **File ▶ Open**, accept the default options, and select **OK** (see [Figure 3-4](#)). IDA will then automatically analyze the file.

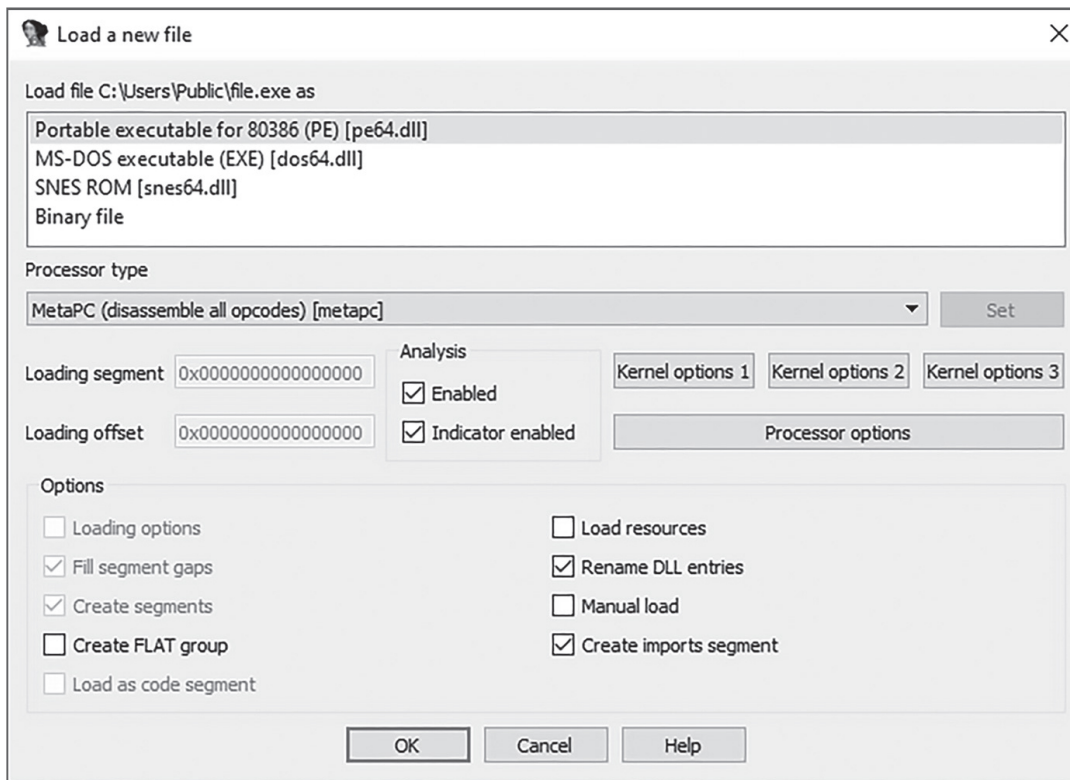


Figure 3-4: Loading a new file into IDA

The IDA interface contains several tabs, some of which represent elements of the file that you may wish to inspect (see [Figure 3-5](#)).

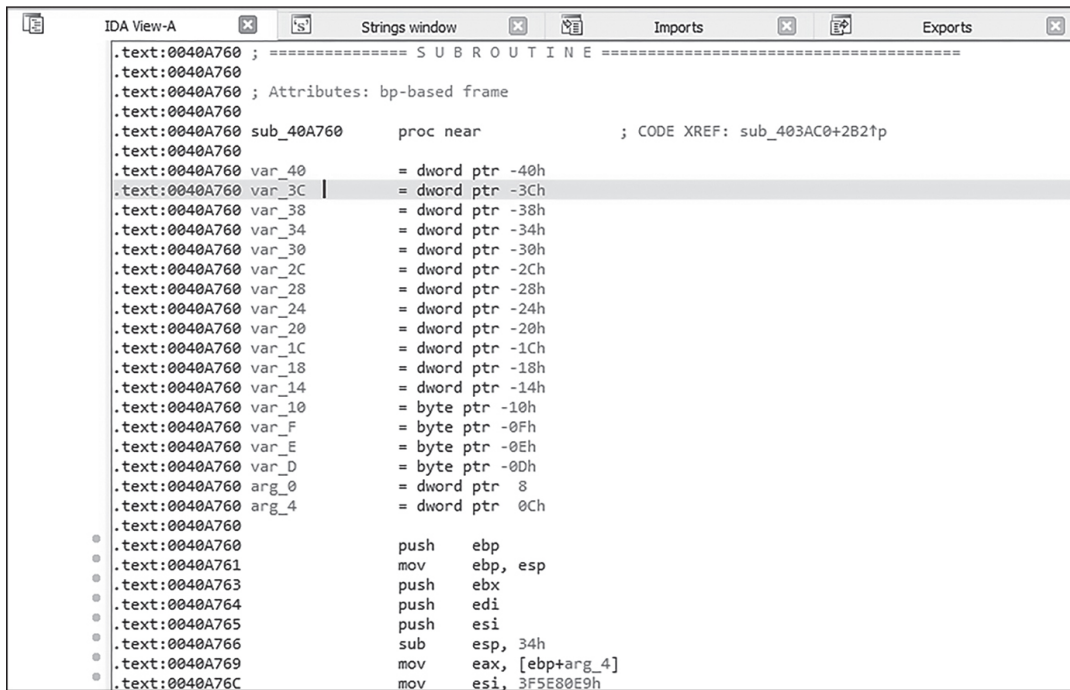


Figure 3-5: The IDA interface

On the Imports tab, you can see several interesting functions, including WinHttp library functions (see [Figure 3-6](#)), which signal that this malware may try to communicate with a server on the internet at some point.

Address	Ordinal	Name	Library
0000000000417194		WinHttpCloseHandle	WINHTTP
0000000000417198		WinHttpConnect	WINHTTP
000000000041719C		WinHttpSendRequest	WINHTTP
00000000004171A0		WinHttpOpen	WINHTTP
00000000004171A4		WinHttpQueryDataAvailable	WINHTTP
00000000004171A8		WinHttpReceiveResponse	WINHTTP
00000000004171AC		WinHttpOpenRequest	WINHTTP
00000000004171B0		WinHttpRequestData	WINHTTP
00000000004171B8		InternetQueryDataAvailable	WININET
00000000004171BC		InternetOpenUrlW	WININET
00000000004171C0		InternetOpenW	WININET
00000000004171C4		InternetCrackUrlW	WININET
00000000004171C8		InternetCloseHandle	WININET
00000000004171CC		InternetReadFile	WININET

Figure 3-6: The list of functions on the IDA Imports tab

The InternetOpenUrlW function can be used by malware to connect to a malicious server on the internet. To inspect this function call in the program, you can simply double-click it in the Imports view and then press CTRL-X to view the cross-references (see [Figure 3-7](#)). Cross-references are addresses in the program's code that contain the selected item.

xrefs to InternetOpenUrlW			
Direction	Typ	Address	Text
Up	r	sub_4023A0+1A9	call ds:InternetOpenUrlW
Up	p	sub_4023A0+1A9	call ds:InternetOpenUrlW

Line 1 of 2

OK Cancel Search Help

Figure 3-7: Cross-references to InternetOpenUrlW

Click **OK** on a cross-reference to jump to the location in the code where the program is calling InternetOpenUrlW, as shown in [Figure 3-8](#).

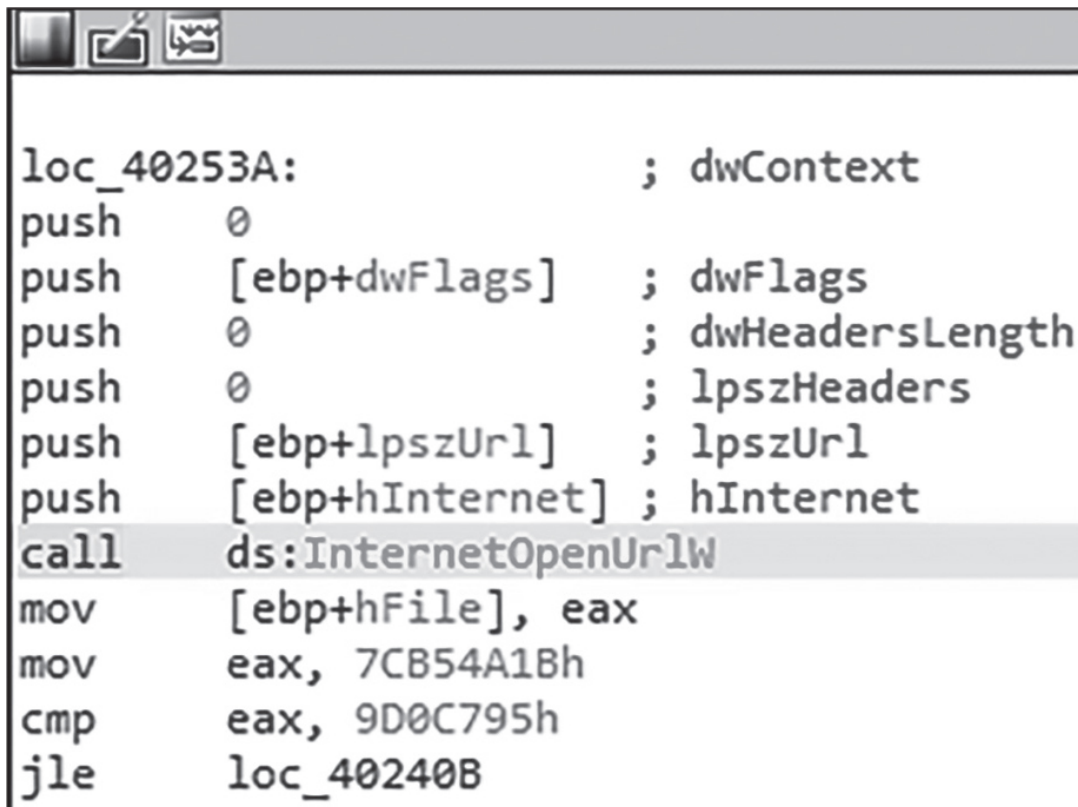


Figure 3-8: The code location for the `InternetOpenUrlW` function call

We're trying to determine what URL is being opened by the malware, but unfortunately, we can't see much in this code, as the input parameters aren't clear. We can see several `push` instructions that are storing parameters on the stack, and one of these parameters is `lpszUrl`, which is the target URL. If we were to run this program, this parameter would be on the stack at address `[ebp+lpszUrl]`. However, since we're only looking at the code statically, there are no parameters on the stack to inspect, which makes our job more difficult.

We could trace the code backward to try to determine what the program is eventually pushing to the stack as a parameter to the `InternetOpenUrlW` function. Sometimes this is valuable, but oftentimes tricky malware obfuscates this data. An alternative method is to load the malware into a debugger and dynamically inspect the stack. We'll take a look at how to do this in a moment. First, let's discuss one more helpful static code analysis tool.

Analyzing with CAPA

CAPA (<https://github.com/mandiant/capa>) is an open source tool written by researchers at Mandiant. Although not a full-fledged disassembler like IDA Pro, it can help malware analysts quickly understand a malware sample's potential behaviors and identify areas of code worth investigating further. CAPA works by scanning a PE file for patterns such as strings and specific assembly instructions. This scan extracts a wealth of information and helps to guide the reverse engineering process. To run CAPA against a target executable file, invoke the following command:

```
> capa.exe malware.exe -vv
```

The `-vv` instruction tells CAPA to deliver extra-verbose information. (Note that `-vvv` returns even more information, and `-v` returns less.)

Figure 3-9 illustrates some sample output from CAPA.

ATT&CK Tactic	ATT&CK Technique
DEFENSE EVASION	Obfuscated Files or Information [T1027] Process Injection::Asynchronous Procedure Call [T1055.004] Process Injection::Process Hollowing [T1055.012] Process Injection [T1055] Virtualization/Sandbox Evasion::System Checks [T1497.001]
DISCOVERY	File and Directory Discovery [T1083] System Information Discovery [T1082]
EXECUTION	Shared Modules [T1129]
CAPABILITY	NAMESPACE
execute anti-VM instructions (2 matches)	anti-analysis/anti-vm/vm-detection
encode data using XOR (2 matches)	data-manipulation/encoding/xor
hash data via WinCrypt	data-manipulation/hashing
initialize hashing via WinCrypt	data-manipulation/hashing
contain a resource (.rsrc) section	executable/pe/section/rsrc
check if file exists	host-interaction/file-system/exists
get hostname	host-interaction/os/hostname
create process	host-interaction/process/create
allocate RWX memory	host-interaction/process/inject
inject APC	host-interaction/process/inject
use process replacement	host-interaction/process/inject
access PEB ldr_data	linking/runtime-linking
parse PE header (4 matches)	load-code/pe

Figure 3-9: CAPA output

This output reveals some interesting information. First, this sample seems to be using evasion techniques such as file obfuscation, process injection, and virtualization and sandbox discovery. These tactics may

be new to you, but don't worry, we'll cover them in great detail throughout this book.

CAPA running in verbose mode will even provide the address where the suspect functionality resides in the executable. **Figure 3-10** shows potential anti-VM instructions identified by CAPA.

```
execute anti-VM instructions (2 matches)
namespace anti-analysis/anti-vm/vm-detection
author moritz.raabe@fireeye.com
scope basic block
att&ck Defense Evasion::Virtualization/Sandbox Evasion::System Checks [T1497.001]
mbc Anti-Behavioral Analysis::Virtual Machine Detection::Instruction Testing [B0009.029]
examples Practical Malware Analysis Lab 17-03.exe_0x401A80
basic block @ 0x140002144
or:
mnemonic: cpuid @ 0x140002157, 0x14000217E
basic block @ 0x140002201
or:
mnemonic: cpuid @ 0x140002203
```

Figure 3-10: Potential anti-VM instructions identified by CAPA

The specific anti-VM instruction in **Figure 3-10** is `cpuid`, an assembly instruction often used by malware to detect a VM environment. CAPA located the addresses in this executable where `cpuid` is called (0x140002157, 0x14000217E, and 0x140002203). Now you can load this malware sample into a disassembler such as IDA Pro and jump to these address locations in an executable to quickly find the `cpuid` instructions.

Figure 3-11 shows another example in which CAPA has identified the addresses where suspected malware resides.

Figure 3-11: CAPA output showing potential process injection

In this case, CAPA has identified a potential process injection technique: APC injection, which is a method malware uses to evade host defenses and stealthily execute malicious code.

Used alongside a disassembler and other static code analysis tools, CAPA can increase the efficiency of the code analysis process and be a

great asset in your malware analysis toolbox. We'll talk more about CAPA in other chapters, but now we'll turn to dynamic code analysis and how it can supplement static code analysis.

While CAPA is incredibly useful, it has a few limitations. First, it has no unpacking or deobfuscation capabilities (at least not at the time of this writing), so in the case of packed and heavily obfuscated malware, CAPA may produce incorrect information or none at all. Second, CAPA occasionally produces false-positive indicators. Always manually verify any functionalities you find with CAPA.

Dynamic Code Analysis

Dynamic code analysis consists of analyzing code while it's actively running, which usually means executing code in a debugger.

Debuggers are similar to disassemblers in that they also disassemble code and present it to you, but they have the added benefit of being able to dynamically execute the code.

The real power of debuggers is that they allow you to set breakpoints on running code. *Breakpoints* are special instructions or flags that trigger an exception (or *break*) in the program that passes control to the debugger itself, allowing you to take control of the running malware sample.

Choosing a Debugger

The x64dbg tool (<https://x64dbg.com>) is a powerful, free, and open source debugger for Windows environments. It's highly customizable and scriptable, and it's supported by the community with many useful plug-ins. Throughout this book, I'll focus specifically on x64dbg, but many debuggers act, look, and feel very similar to it. Some alternatives to x64dbg are the much older OllyDbg, IDA Pro's built-in debugger, or WinDbg.

NOTE

There are technically two versions of x64dbg: x32dbg (for debugging 32-bit programs) and x64dbg (for 64-bit programs). They function exactly the same but focus on different architectures. I'll refer to the program as x64dbg, as the debugger's creator does, but just remember that in order to debug a 32-bit (x86) program, you must use the x32dbg version.

Starting a Debugging Session in x64dbg

To load an executable file into x64dbg, choose the right version of x64dbg (the 32-bit or 64-bit version) and select **File ▶ Open**. (If you use the wrong version, you'll get a helpful message at the bottom of the debugger window, such as "Use x32dbg to debug this file!".) Alternatively, you can attach the debugger to a currently running malware process by selecting **File ▶ Attach**. This approach has the downside that you might miss critical behaviors that occurred before you were able to attach to the process and begin debugging it.

Once a program is loaded into a debugger, it's running as a child process under the debugger. In most situations, the debugger calls the function `DebugActiveProcess`, which permits it to attach to the active process and begin the debug session. We'll return to this API call in [Chapter 10](#). For now, let's take a look at the most important sections of the x64dbg user interface, shown in [Figure 3-12](#).

Figure 3-12: The x64dbg debugger

The CPU Tab

The CPU pane at the top left of the x64dbg window lists the instructions the malware will execute, or has already executed, in sequential order. In this window, you can step through the code line by line or skip ahead to the more interesting parts. EIP (or RIP for x64 programs)

marks the instruction that will be executed next, as shown in **Figure 3-13**.

Figure 3-13: The x64dbg CPU tab

The CPU Registers Pane

At the top right of the x64dbg window, you'll see the CPU Registers pane (see **Figure 3-14**). This pane displays each register and flag and its currently stored value. This is helpful for keeping track of data and addresses stored in registers.

Figure 3-14: The x64dbg CPU Registers pane

You might also have noticed the EFLAGS section of this pane, which displays the flag registers and their values.

The Arguments Pane

The Arguments pane (see **Figure 3-15**) is positioned in the middle right of the x64dbg window.

Figure 3-15: The x64dbg Arguments pane

This pane displays the list of arguments on the stack that are passed to the current function call in the program. This information is invaluable for monitoring and altering arguments to functions.

The Stack Pane

On the bottom right of the x64dbg window is the Stack pane. This pane displays the currently running thread's stack memory (see **Figure 3-**

16).

Figure 3-16: The x64dbg Stack pane

It is helpful to reference this stack pane when the malware calls a function (whether an internal function or a Windows API function). The parameters that the malware passes to the function being called will oftentimes be pushed to the stack prior to the function call, especially in 32-bit malware.

The Dump Pane

At the bottom left is the Dump pane, shown in **Figure 3-17**.

Figure 3-17: The x64dbg Dump pane

This view allows you to inspect and monitor memory addresses (or *dumps*) dynamically. You can also set *watches* to have x64dbg notify you when a specific event occurs, such as when a specific register is modified.

The Memory Map Tab

Finally, the Memory Map tab can be accessed from the series of tabs toward the top of the x64dbg window. It's very useful during dynamic code analysis, as it displays the program's virtual memory layout and allows you to dig deeper into each memory region (see **Figure 3-18**).

Figure 3-18: The x64dbg Memory Map tab

One of the most important uses of the Memory Map pane is hunting for executable code in memory during the malware unpacking. I'll

cover this in more detail, and memory in general, in [Chapter 17](#).

Analyzing with x64dbg

Next, we'll look at a typical debugging scenario for a malware sample to give you a high-level overview of what dynamic code analysis inside a debugger looks like. We'll be analyzing the same file we used in "Analyzing with IDA" on [page 53](#).

Disabling ASLR

As [Chapter 1](#) explained, address space layout randomization loads your malware's executable and libraries into randomized memory locations. Although it's an effective way to thwart attackers, it will also hamper your dynamic code analysis efforts, so you should disable it. To disable ASLR for this file, you have a few options, but you could use CFF Explorer (<https://ntcore.com>) here because it makes this process quick and easy. Simply load the malware sample into CFF Explorer, select the **Optional Header** category on the left menu, and click the **DllCharacteristics** box (see [Figure 3-19](#)).

Figure 3-19: Setting a file's DLL characteristics in CFF Explorer

You may recognize the Optional Header from [Chapter 1](#). One of the fields inside the Optional Header, the `DllCharacteristics` field, contains a number of attributes for the executable file.

Next, in the pop-up menu, uncheck the box next to **DLL Can Move**, as shown in [Figure 3-20](#).

Figure 3-20: Disabling ASLR in CFF Explorer

Finally, click **OK** and remember to save the modified file by selecting **File ▶ Save**.

Running the Code

Now load the file into x64dbg (more specifically, x32dbg, since this is a 32-bit file) by selecting **File ▶ Open**. Once that's done, you should see several options on the Debug menu for running and debugging the program (see [Figure 3-21](#)).

Figure 3-21: The Debug menu in x64dbg

Let's go through the options in order:

Run

Runs the program until something stops the code, such as an exception, error, or breakpoint, or a process termination or exit.

Run Until Selection

Runs the program until it hits an address in the code that you have manually selected.

Step Into

Allows you to step into a function about to be executed in order to manually debug and inspect it. This option is commonly known just as a *step* and will be discussed frequently throughout this book. Its keyboard shortcut is F7.

Step Over

Allows you to step over a function, skipping its execution entirely. This is a good way to save time and skip over code that you're not inter-

ested in analyzing.

Execute Till Return

Executes the program until it hits the next return (`ret`) instruction.

Run to User Code

Executes the program and breaks on the user code. This is one of the most useful debugging options, as we'll discuss shortly.

There are more debugging options in the Advanced menu. For example, the option Run (Swallow Exceptions) forces the debugger to essentially ignore exceptions. Since a large number of exceptions can be problematic while debugging (and malware can even purposefully generate exceptions to annoy you!), this can be a good option to save you some frustration.

Once the file has been loaded into the debugger, it must complete initialization tasks such as loading required libraries and other Windows operating system tasks. We're not interested in this for our purposes and would instead like to skip ahead to the malware file's entry point (the part of the code we want to analyze). To do this, select

Debug ▶ Run to User Code (see [Figure 3-22](#)).

Figure 3-22: The Run to User Code entry point

Now that the malware has been loaded into the debugger, we could simply start executing and stepping through code. This isn't usually the most efficient method, however. It's better to have an idea in mind of what code we'd like to inspect in the malware. In the previous section, we found something of interest in IDA: the `InternetOpenUrlW` function. Let's find and inspect this area of code.

Using Software Breakpoints

You set software breakpoints by inserting special CPU instructions, such as `INT 3` (in hexadecimal, `0xCC`), which is the most common breakpoint instruction, or `INT 2D` (in hexadecimal, `0xCD 0x2D`). Keep in mind that creating software breakpoints directly modifies the running program's code. Most benign programs don't care about this and ignore it. However, some malware doesn't want to be debugged and will attempt to detect and circumvent your breakpoints.

To jump to the area of code that executes the `InternetOpenUrlW` function, you can simply set a breakpoint on this function call. The most efficient way to do this is by entering this instruction in the Command bar at the bottom of the x64dbg window:

```
bp InternetOpenUrlW
```

See [Figure 3-23](#).

Figure 3-23: Setting a breakpoint on a function call

Next, execute the sample by using the **Debug ▶ Run** command (or pressing F9 on the keyboard). This will execute the malware and break on our target function `InternetOpenUrlW`, as shown in [Figure 3-24](#).

Figure 3-24: Breakpoint hit!

If you inspect the parameters in the Arguments pane, you should see a full URL on the stack (see [Figure 3-25](#)).

Figure 3-25: A URL on the stack

The internet service *iplogger.org* can be used to log and track IP addresses. The malware is likely using this service to track the hosts that are infected by this malware sample or to obtain the victim's IP address.

Software breakpoints aren't limited to function calls; you can set them on any address you choose. In the CPU pane of x64dbg, simply right-click on an address where you'd like to set a breakpoint and select **Breakpoint ► Toggle** (or press F2). Once the instructions at this address are about to be executed by the CPU, the debugger will pause.

Setting Hardware and Memory Breakpoints

As an alternative to software breakpoints, you can set *hardware breakpoints*, which are implemented by the CPU itself, or *memory breakpoints*, which are implemented using memory protections. Hardware breakpoints are set and stored in CPU registers, specifically DR0, DR1, DR2, and DR3. When you set a hardware breakpoint, the address where the breakpoint was set is stored in one of these DR registers. Hardware breakpoints have the advantage that they do not directly modify code, so they are less invasive and thus a bit more difficult for malware to detect. Their primary disadvantage is that only four of them can be placed at a time due to the limited number of DR registers.

Memory breakpoints modify the protection attributes of a memory page, effectively causing an exception when that memory page is accessed. This is usually accomplished by modifying the memory page's `PAGE_GUARD` attribute. Memory breakpoints are particularly useful for monitoring addresses in memory. For example, if you identify an interesting string in memory at runtime (such as a URL or filename), setting a memory breakpoint on that string's address can help you determine where and how the malware is using that string. One downside to memory breakpoints is that, since they directly modify memory page protections, they can interfere with a program's operations.

Specifically, memory breakpoints may cause a program to crash when it attempts to allocate new memory pages or modify existing pages. Once a memory breakpoint is triggered, the memory protections for the page are reset to the way they were before the breakpoint was set. So, if a malware sample modifies the protections of the page before or after the breakpoint is triggered, you may undo the malware's changes or the malware may inadvertently (or purposely!) undo your breakpoint. Always be cautious when using memory breakpoints. Since hardware and memory breakpoints are often grouped together, from now on I'll use the term *hardware breakpoint* to refer to both.

There are multiple ways to set a hardware breakpoint in a program being debugged:

- To set a hardware breakpoint on an address in the CPU pane, right-click the address and then click **Breakpoint ▶ Set Hardware on Execution**.
- In the Dump pane view, select a single byte for which to set a breakpoint, right-click it, and mouse over **Breakpoint** to select from several options.
- In the Memory Map tab view, highlight a memory region where you'd like to set a breakpoint. Right-click it and select **Memory Breakpoint**, then select the breakpoint option.

Table 3-9 outlines various types of hardware and memory breakpoints and ways to implement them.

Table 3-9: Types of Hardware Breakpoints in x64dbg

Breakpoint type	Description
Hardware, Access	Set a hardware breakpoint on access. When this address is accessed in any way (read, written to, or executed), the breakpoint will be triggered.
Hardware, Write	Set a hardware breakpoint on write. When this address is about to be written to, the breakpoint will be triggered.
Hardware, Execute	Set a hardware breakpoint on execute. When the instructions at this address are about to be executed, the breakpoint will be triggered.
Memory, Access	Set a memory breakpoint on access. When this memory page is accessed in any way (read, written to, or executed), the breakpoint will be triggered.
Memory, Read	Set a memory breakpoint on read. When this memory page is about to be read from, the breakpoint will be triggered.
Memory, Write	Set a memory breakpoint on write. When this memory page is about to be written to, the breakpoint will be triggered.
Memory, Execute	Set a memory breakpoint on execute. When instructions in this memory page are about to be executed, the breakpoint will be triggered.

It's important to note that hardware breakpoints can be set on a byte, a word, or a dword. Setting a hardware breakpoint on a specific byte, for example, will trigger an exception when that specific byte is accessed. For words and dwords, the exception will trigger if the entire word (2 bytes) or dword (4 bytes) is accessed. Hardware breakpoints can also be set in two modes: singleshoot and restore. The *singleshoot breakpoint*, once triggered, is removed and will not trigger again. The *restore breakpoint* will restore itself once triggered, creating a persistent breakpoint that will trigger again if the specific address is accessed again.

For malware analysis specifically, hardware breakpoints are most often used to counter common debugger and breakpoint detection techniques, as well as during manual unpacking of a sample. We'll discuss hardware breakpoints specifically in this context in [Chapters 10](#) and [17](#).

Patching and Modifying Code

Patching means modifying or removing instructions from a program. To do this in x64dbg, right-click the address of the code you need to modify and select **Binary**, as shown in [Figure 3-26](#).

Figure 3-26: Editing and patching code in x64dbg

The Edit option allows you to modify the code at this address. The Fill with NOPs option is a good way to quickly clear code; it fills this memory address with `nop` instructions, essentially telling the program to skip over this section of code. To patch out the `call` to `InternetOpenUrlW` in the current malware sample, for example, you'd highlight the line that contains the function call instruction (`call InternetOpenUrlW`) and then fill it with NOPs.

In this scenario, there's likely no point in patching out the call instruction to `InternetOpenUrlW` (unless you want to prohibit the malware from connecting to the internet). In general, however, patching code in a running malware sample can be a very powerful way to bypass anti-analysis and evasion techniques and control a malware's flow of execution.

Tracing API Calls with API Monitor

API Monitor (<http://www.rohitab.com/downloads>) is a great debugging tool to add to your toolbox, as it allows you to trace and monitor malware API function calls. It also enables you to set breakpoints on specific functions, so it acts as a rudimentary debugger as well.

You can select the APIs and functions that you'd like to trace in the API Filter window (at the top left of the API Monitor window), as shown in [Figure 3-27](#).

Figure 3-27: The API Filter menu in API Monitor

Here I've selected all of the internet- and network-related functions under the `Wininet.dll` library.

To monitor a new process, click the **Monitor New Process** button in the middle of the API Monitor window and then select the malware executable you want to analyze. The default options are good enough, so click **OK** (see [Figure 3-28](#)).

Figure 3-28: Monitoring a new process in API Monitor

After the malware executes and runs for a few minutes, some API calls will begin to populate the Summary window, as you can see in [Figure 3-29](#).

Figure 3-29: A list of API calls in API Monitor

You may recognize some of these calls from our previous analysis in x64dbg. The power of API Monitor is that it lets you quickly see the function calls you're interested in as well as their arguments and return values. This is immensely valuable for getting a quick read on a malware sample's capabilities or for monitoring and tracing suspect behaviors.

STUCK IN THE RABBIT HOLE

Just as it's important to know how and when to conduct deeper code analysis, it's important to know when to stop. You can quickly go so far down the rabbit hole of reverse engineering that you get stuck in a code maze you may never get out of. An experienced reverser can recognize these rabbit holes and quickly escape to the surface; a novice, however, may not realize when they're truly lost or analyzing code that's not worth investigating.

Remember your goals for malware analysis and reversing. If you're simply attempting to get an understanding of a malware's capabilities and behaviors at a high level to quickly assess the malware's impact on your organization, then deeper code analysis may not be necessary. Always consider doing a cost-benefit analysis. If spending another day stuck in code won't help you achieve your goals, then maybe it isn't necessary. That said, if you're simply interested in getting better at reverse engineering, then by all means, carry on!

Summary

In this chapter, you took a crash course in assembly code and explored the static and dynamic code analysis processes, including the role of disassemblers and debuggers, at a high level. In a typical scenario,

you'd use static code analysis to identify and analyze code worth investigating further, and then you'd follow up with dynamic code analysis in a debugger. When you're reverse engineering malware, it can be helpful to switch between static and dynamic code analysis often to fully understand the code you're investigating.

This chapter concludes the foundational topics of malware analysis. In the next few chapters, we'll start to dig into how evasive malware is able to detect virtual machines, sandboxes, and the tools analysts use to investigate malware internals.